

Rerooting Dynamic Programming

Sean

October 4, 2026

Outline

- Distance sums for every vertex
- Two passes and an implementation
- Farthest distances and child exclusion
- Combining neighbor contributions



What does rerooting mean?

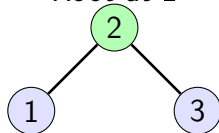
Rerooting means choosing a different root for the same tree.

Root at 1



1 is the parent of 2.
2 is the parent of 3.

Root at 2



2 is the parent of both
1 and 3.

An answer for every root

- Tree DP usually starts by choosing one root
- Some problems ask for the result at *every* possible root
- Adjacent roots share almost all of the same work

Rerooting DP

Reuse information across each edge to compute all root answers.

1 Distance sums

2 Two passes

3 Farthest distances

4 Neighbor contributions

Distance sums

Example: Tree Distances II

CSES: Tree Distances II

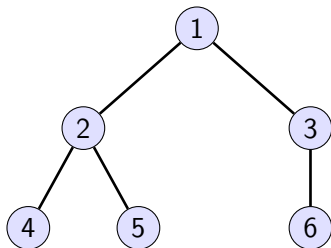
For each vertex of an unweighted tree, find the sum of its distances to all vertices.

$$\text{ans}[u] = \sum_{x=1}^n \text{dist}(u, x)$$

- $1 \leq n \leq 2 \cdot 10^5$
- Distance counts edges, and $\text{dist}(u, u) = 0$



One tree, six answers



From vertex 1:

$$\begin{aligned} 0 + 1 + 1 \\ + 2 + 2 + 2 = 8 \end{aligned}$$

u	1	2	3	4	5	6
$\text{ans}[u]$	8	8	10	12	12	14

Repeating the same traversal

- A DFS or BFS from one vertex finds its distance sum in $O(n)$
- Running it from every vertex costs $O(n^2)$
- Moving from u to a neighbor v changes every distance by exactly 1

The useful question

Which vertices become closer, and which become farther away?



Subtree information at a fixed root

Root the tree at 1. For every vertex u , store:

- $\text{sz}[u]$: number of vertices in its subtree
- $\text{sub}[u]$: sum of distances from u to vertices in its subtree

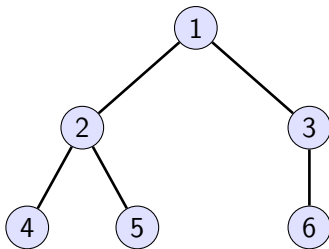
For the children v of u :

$$\text{sz}[u] = 1 + \sum_v \text{sz}[v]$$

$$\text{sub}[u] = \sum_v (\text{sub}[v] + \text{sz}[v])$$



The first answer

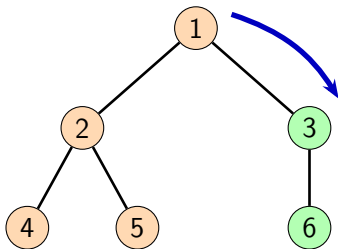


u	$sz[u]$	$sub[u]$
4, 5, 6	1	0
2	3	2
3	2	1
1	6	8

$$sub[1] = (2+3) + (1+2)$$

$$ans[1] = 8$$

Moving the root across one edge



Move from 1 to 3.

- 2 vertices: distance decreases by 1
- 4 vertices: distance increases by 1

$$\text{ans}[3] = 8 - 2 + 4 = 10$$

The rerooting transition

Let v be a child of u in the tree rooted at 1.

- $\text{sz}[v]$ vertices become one edge closer
- $n - \text{sz}[v]$ vertices become one edge farther

$$\text{ans}[v] = \text{ans}[u] - \text{sz}[v] + (n - \text{sz}[v])$$

One constant-time update

$$\text{ans}[v] = \text{ans}[u] + n - 2 \text{sz}[v]$$



A complete rerooting trace

Begin with $\text{ans}[1] = 8$ and $n = 6$.

Move	$\text{sz}[v]$	Change $6 - 2 \text{sz}[v]$	Answer
$1 \rightarrow 2$	3	0	8
$1 \rightarrow 3$	2	+2	10
$2 \rightarrow 4$	1	+4	12
$2 \rightarrow 5$	1	+4	12
$3 \rightarrow 6$	1	+4	14

Subtree sizes stay tied to the original root.



1 Distance sums

2 Two passes

3 Farthest distances

4 Neighbor contributions

Two passes

Children first, then parents first

- 1 Process children before their parent to build `sz` and `sub`
- 2 Set `ans[1] = sub[1]`
- 3 Process each parent before its children to transfer the answer across every edge

A fixed traversal root

Keep the same parent array in both passes. Rerooting changes the answer's viewpoint.



C++: a parent-before-child order

Input: $n \geq 1$ and an undirected adjacency list g , indexed 1 to n .

```
vector<int> parent(n + 1, 0), order{1};  
parent[1] = -1;  
for (int i = 0; i < (int)order.size(); ++i) {  
    int u = order[i];  
    for (int v : g[u]) {  
        if (v == parent[u]) continue;  
        parent[v] = u;  
        order.push_back(v);  
    }  
}
```

Reversing order puts every child before its parent.



C++: subtree sizes and sums

```
vector<int> sz(n + 1, 1);  
vector<long long> sub(n + 1, 0), ans(n + 1);  
for (int i = n - 1; i > 0; --i) {  
    int u = order[i], p = parent[u];  
    sz[p] += sz[u];  
    sub[p] += sub[u] + sz[u];  
}  
ans[1] = sub[1];
```

Each subtree contributes its internal distances plus one edge for every vertex it contains.



C++: answers for every vertex

```
for (int i = 1; i < n; ++i) {  
    int u = order[i], p = parent[u];  
    ans[u] = ans[p] + n - 2LL * sz[u];  
}  
for (int u = 1; u <= n; ++u)  
    cout << ans[u] << (u == n ? '\n' : ' ');
```

- $O(n)$ time and $O(n)$ memory, including the graph
- long long stores sums as large as $n(n - 1)/2$
- Iterative passes also handle a chain of 200 000 vertices



Why the two passes are correct

- In reverse order, all child subtrees are ready before they contribute to their parent
- The root's subtree is the whole tree, so its first answer is correct
- The edge transition accounts for every vertex exactly once
- Forward order therefore gives a correct answer to each child of a vertex with a correct answer



Extension: weighted distances

If edge (u, v) has length w , moving across it changes each distance by w .

$$\text{ans}[v] = \text{ans}[u] + w(n - 2\text{sz}[v])$$

- Build the first sum using weighted distances
- For $\sum_x c[x] \text{dist}(u, x)$, replace subtree size with subtree weight and n with total vertex weight
- Choose integer types from the bounds on the full weighted sum



1 Distance sums

2 Two passes

3 Farthest distances

4 Neighbor contributions

Farthest distances

Example: Tree Distances I

CSES: Tree Distances I

For every vertex, find the greatest distance to any vertex in the tree.

$$\text{far}[u] = \max_x \text{dist}(u, x)$$

- The earlier tree lecture used diameter endpoints
- Rerooting gives another $O(n)$ solution
- This example introduces a common issue: excluding one child's contribution



Distances inside and outside a subtree

With the traversal root fixed at 1:

- $\text{down}[u]$: greatest distance to a vertex in u 's subtree
- $\text{up}[u]$: greatest distance to a vertex outside that subtree, or 0 when there is none

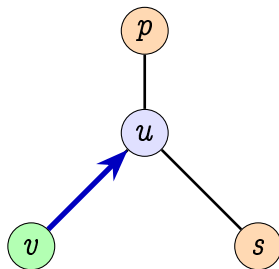
$$\text{down}[u] = \max(0, \max_{v \text{ child of } u} (1 + \text{down}[v]))$$

$$\text{far}[u] = \max(\text{down}[u], \text{up}[u])$$

Leaf: $\text{down} = 0$. Root: $\text{up}[1] = 0$. All distances count edges.



The route through the parent



From v , take one edge to u , then:

- Stop at u
- Continue outside u 's subtree
- Enter a sibling's subtree

Excluding the child being updated

For a child v of u :

$$\text{up}[v] = 1 + \max \left(0, \text{up}[u], \max_{\substack{s \text{ child of } u \\ s \neq v}} (1 + \text{down}[s]) \right)$$

- Reusing $\text{down}[u]$ directly may send the path back into v 's subtree
- Scanning every sibling for every child costs $O(n^2)$ on a star
- The two largest child contributions give each exclusion in $O(1)$



Two largest contributions, including ties

Suppose three children contribute distances 4, 4, 2 at u .

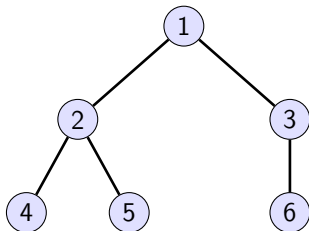
Excluded contribution	Remaining	Best remaining
First 4	4, 2	4
Second 4	4, 2	4
2	4, 4	4

Ties matter

Keep the two largest values with duplicates: 4 and 4.



Farthest-distance trace



u	down	up	far
1	2	0	2
2	1	3	3
3	1	3	3
4	0	4	4
5	0	4	4
6	0	4	4

For vertex 2, the parent side reaches vertex 6 in 3 edges.

C++: downward distances

Reuse parent and order from the first example.

```
vector<int> down(n + 1, 0), up(n + 1, 0);  
vector<int> far(n + 1, 0);  
for (int i = n - 1; i > 0; --i) {  
    int u = order[i], p = parent[u];  
    down[p] = max(down[p], 1 + down[u]);  
}
```

The reverse pass prepares every child contribution before its parent needs it.



C++: the two largest child values

The forward loop continues on the next slide.

```
for (int u : order) {  
    int best1 = 0, best2 = 0;  
    for (int v : g[u]) if (parent[v] == u) {  
        int x = 1 + down[v];  
        if (x >= best1) {  
            best2 = best1;  
            best1 = x;  
        } else {  
            best2 = max(best2, x);  
        }  
    }  
}
```

The `>=` branch preserves two equal maximum values.



C++: the parent-side contribution

Continuation of the same forward loop.

```
far[u] = max(down[u], up[u]);  
for (int v : g[u]) if (parent[v] == u) {  
    int x = 1 + down[v];  
    int sibling = (x == best1 ? best2 : best1);  
    up[v] = 1 + max(up[u], sibling);  
}  
}
```

- Zero also represents stopping at u
- Each adjacency list is scanned a constant number of times
- Total: $O(n)$ time and $O(n)$ memory

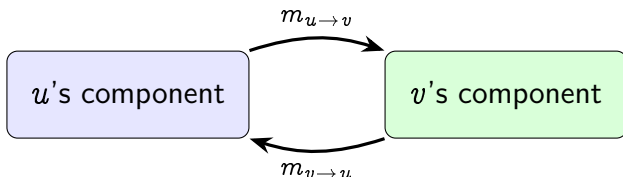


- 1 Distance sums
- 2 Two passes
- 3 Farthest distances
- 4 Neighbor contributions

Neighbor contributions

A state for each direction of an edge

Remove edge (u, v) . It splits the tree into two components.



- $m_{u \rightarrow v}$ summarizes the component containing u
- To compute it, combine information entering u from every neighbor except v



Why two passes are enough

Choose vertex 1 as the traversal root.

- 1 In reverse order, compute every child-to-parent message
- 2 In forward order, the parent message and all child messages at u are ready
- 3 Combine all neighbors for the answer at u , and all except v for the message to child v

The remaining task

Compute each “all except one” combination efficiently.



Excluding one contribution

Combination	Exclusion method
Sum	Subtract the excluded value
Maximum	Keep the two largest values
Associative merge	Prefix and suffix combinations

- A constant-time merge gives $O(\deg(u))$ work at vertex u
- $\sum_u \deg(u) = 2(n - 1)$, so the total is $O(n)$



Associative merges and empty groups

Associative means grouping does not change the result:

$$(a \oplus b) \oplus c = a \oplus (b \oplus c).$$

An **identity** e represents an empty group:

$$e \oplus a = a \oplus e = a.$$

Merge	Operation	Identity
Sum	+	0
Product	$\times$	1

Grouping may change. The order of contributions stays fixed.



Prefix and suffix combinations

Neighbor contributions at u : $a_0, a_1, \dots, a_{d-1}$.

Let $\oplus$ be associative with identity e .

$$P_0 = e, \quad P_{i+1} = P_i \oplus a_i$$

$$S_d = e, \quad S_i = a_i \oplus S_{i+1}$$

All contributions except a_i

$$P_i \oplus S_{i+1}$$

Include the parent-side contribution too. Preserve neighbor order if the merge is not commutative.



Prefix and suffix example

With addition, let $a = [2, 5, 3, 4]$ and exclude $a_1 = 5$.

Boundary i	0	1	2	3	4
P_i	0	2	7	10	14
S_i	14	12	7	4	0

$$P_1 + S_2 = 2 + 7 = 9$$

- Build both arrays once in $O(d)$
- Each exclusion then takes $O(1)$
- Also works for products and other associative merges



Practice extension: connected sets

AtCoder Educational DP Contest: V – Subtree

For each vertex u , count connected vertex sets containing u , modulo M .

- $1 \leq n \leq 10^5$ and $2 \leq M \leq 10^9$
- Every selected vertex must connect to u through selected vertices
- A neighbor's branch can contribute nothing, or a connected set containing that neighbor



One independent choice per branch

Let $N(u)$ be the neighbors of u . The message $m_{u \rightarrow v}$ counts connected sets containing u on u 's side of edge (u, v) .

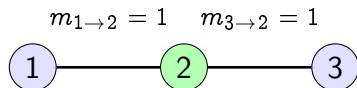
$$m_{u \rightarrow v} = \prod_{w \in N(u) \setminus \{v\}} (1 + m_{w \rightarrow u}) \pmod{M}$$

$$\text{ways}[u] = \prod_{w \in N(u)} (1 + m_{w \rightarrow u}) \pmod{M}$$

- The 1 means choosing no vertex from that branch
- An empty product is 1: the set containing only u
- Prefix/suffix products exclude each branch in constant time



Connected sets on a three-vertex path



$$\text{ways}[2] = (1 + 1)(1 + 1) = 4$$

The four sets are $\{2\}$, $\{1, 2\}$, $\{2, 3\}$, and $\{1, 2, 3\}$.

$$m_{2 \rightarrow 1} = 2, \quad \text{ways}[1] = 1 + 2 = 3$$

Values above are before taking the modulus. By symmetry, $\text{ways}[3] = 3$.

C++: products without division

At vertex u , a contains every neighbor factor $(1 + m) \bmod M$.

```
int d = (int)a.size();
vector<long long> pref(d + 1, 1), suff(d + 1, 1);
for (int i = 0; i < d; ++i)
    pref[i + 1] = pref[i] * a[i] % M;
for (int i = d - 1; i >= 0; --i)
    suff[i] = a[i] * suff[i + 1] % M;
for (int i = 0; i < d; ++i) {
    long long without = pref[i] * suff[i + 1] % M;
    // Message from u to neighbor i is "without".
}
```

M can be composite and factors can be zero. Division may have no modular inverse.



Common mistakes

- Changing subtree sizes halfway through the distance-sum pass
- Sending a child information that still includes its own branch
- Losing duplicate maxima when keeping the two best values
- Rebuilding an exclusion with a full scan for each neighbor
- Forgetting the empty merge, $n = 1$, or a wide enough integer type



Practice roadmap

- CSES: Tree Distances II
Derive and implement the distance-sum transition
- CSES: Tree Distances I
Compute farthest distances with child exclusion
- AtCoder DP: V – Subtree
Combine neighbor contributions with prefix/suffix products



Rerooting checklist

- 1 Define what a subtree or directed-edge state means
- 2 Work out how neighboring contributions combine
- 3 Compute the child-side information first
- 4 Pass the parent-side information down, excluding the receiving child
- 5 Check the total work across all adjacency lists

The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

